

L4

模型微调与 私有化大模型

5. 大模型量化方法

大模型量化原理

什么是模型量化 (Quantization)



定义

- 将神经网络中的浮点数 (FP16/FP32) 参数或激活值, 用更少位宽 (如 INT8 / INT4) 表示的过程,
- 以 **减小模型体积、降低显存占用、加快推理速度**,
- 同时尽量保持精度不变。

核心思想

量化: $q = \text{round}\left(\frac{x - \min}{s}\right)$, 反量化: $x' = S \times q + \min$

- **x**: 原始浮点数
- **q**: 量化后的整数
- **S (scale)**: 缩放系数
- **x'**: 反量化后的近似浮点值

- 将连续的浮点区间映射到有限的离散整数范围内。例如：FP32 值 $\in [-1.0, 1.0] \rightarrow \text{INT8} \in [-128, 127]$
- 量化： $q = \text{round}\left(\frac{x - \min}{s}\right)$ ，反量化： $x' = S \times q + \min$

原始浮点值 x	INT8量化后 q	反量化后 $\hat{x}$	误差
-1.0	-128	-1.00	0.00
-0.5	-64	-0.50	0.00
0.0	0	0.00	0.00
0.5	64	0.50	0.00
0.9	115	0.898	0.002

为什么要量化 (Why Quantize)



- **① 模型太大**
 - 现代大语言模型参数量巨大（如 Qwen-14B、LLaMA2-70B）。
 - FP16 模型常需几十至上百GB显存，普通设备无法运行。
- **② 推理太慢**
 - 浮点计算复杂、功耗高；
 - 整数运算能在硬件上更高效并行。
- **③ 部署太贵**
 - 云端GPU成本高；
 - 边缘设备资源有限。
- **✓ 量化的目标：**
 - “让大模型在有限硬件上高效运行。”

为什么要量化-模型太大



模型	参数量	FP16 模型大小
Qwen1.5-1.8B	1.8 B	≈ 7 GB
LLaMA2-13B	13 B	≈ 52 GB
Qwen-14B	14 B	≈ 56 GB
LLaMA2-70B	70 B	≈ 280 GB

- FP16（半精度浮点）每个参数需要 2 字节，而 FP32 则需要 4 字节。
- 一个几十亿参数的模型，仅存放参数就需要几十 GB 显存或内存，普通 GPU（如 RTX 3060 12GB）根本无法加载。

精度	每参数位数	大小缩减倍数
FP16	16 bit	1×
INT8	8 bit	2× 更小
INT4	4 bit	4× 更小

原因：

- 浮点运算（FP16 / FP32）需要复杂的乘加电路；
- GPU/CPU 对整数矩阵乘法（INT8 / INT4）有专门的加速指令（如 NVIDIA Tensor Core INT8、ARM dot product、Intel VNNI 等）；
- 因此整数推理可获得更高的吞吐与更低的延迟。

量化类型	运算类型	推理加速比	能耗降低
FP16	浮点	1×	baseline
INT8	整数	1.5 – 2×	↓ 30 – 40 %
INT4	整数	2 – 3×	↓ 50 % 以上

问题：

- 云端 GPU（如 A100/H100）租金昂贵；
- 每增加一台 GPU 成本线性增长；
- 边缘端（如 Jetson、手机、NPU）资源有限，无法支持大模型 FP16 运行。

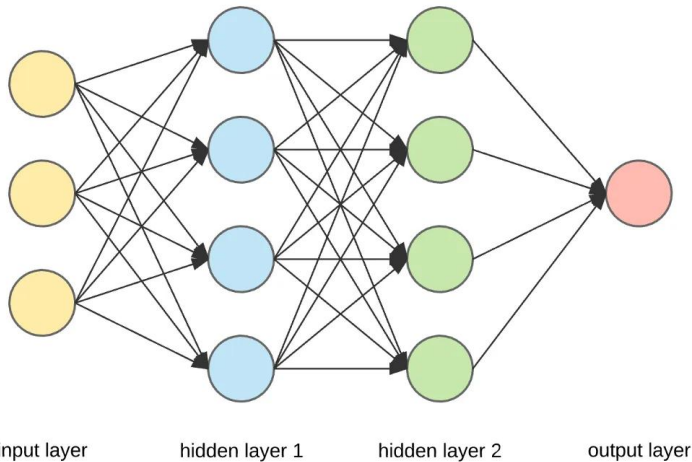
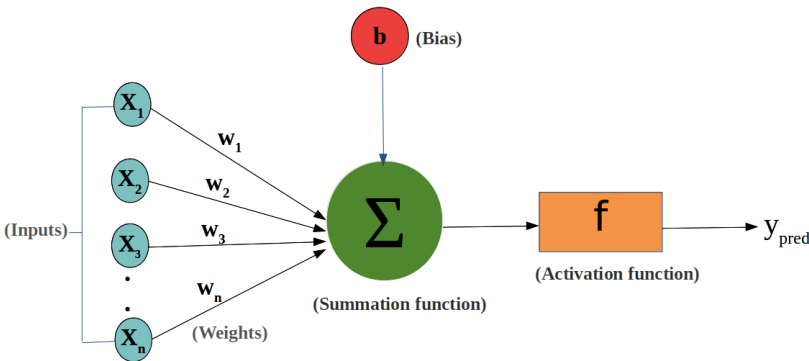


量化价值：

- 显存减半 → 同一硬件可运行更大模型；
- 运算减半 → 更低功耗、更高并发；
- 成本下降 → 云端推理可用更便宜 GPU（如 T4、L4）；
- 轻量设备 → 可在边缘/本地实现 AI 推理。

神经网络中两个最常被量化的对象：

- 1.权重 (Weights) — 模型参数
- 2.激活值 (Activations) — 每层输出



方法	方法类别	核心思想	技术特点	适用场景
GPTQ	推理量化（误差优化）	事后补救：Hessian近似最小化输出误差	精度高、逐层优化、计算较慢	GPU服务器部署、离线高精度压缩
AWQ	推理量化（激活感知）	事前保护：基于激活分布调整scale	无需训练、速度快、工程稳定	GPU推理、边缘设备部署（主流方案）
SmoothQuant	激活+权重量化	将激活难度转移到权重（平滑分布）	支持W8A8、硬件友好	高性能推理加速（数据中心）
GGUF	文件格式 / 推理量化	分块量化 + 混合位宽	优化存储与CPU推理效率	CPU、本地部署（如 llama.cpp 生态）
QAT	训练感知量化	在训练中模拟量化误差	精度最高，但训练成本高	工业级高精度模型
QLoRA	微调方法	量化基座 + LoRA低秩训练	显存占用低、训练高效	消费级GPU微调

大模型量化方法介绍

原理

在模型训练完成后，不再重新训练，仅使用**少量校准数据**，来估计每一层激活值与权重的范围 (min/max)，然后将浮点数映射为整数。

$$q = \text{round}\left(\frac{x - \min}{S}\right), S = \frac{\max - \min}{2^b - 1}$$

优点

- **不需要重新训练**，快速且易实现；
- 只需少量样本；
- 在小模型或CV任务中效果良好。

工作流程

① 模型准备

加载训练完成的浮点模型 (FP16 / FP32) 。

② 校准

使用一小部分代表性数据样本 (通常几百条) 通过模型前向推理, 统计每一层权重与激活的 **最大最小值 / 均方差分布**。

目的: 找到合适的量化 scale, 使映射误差最小。

③ 量化映射

将浮点参数转换为整数 (INT8/INT4) , 并记录对应 scale 与 zero_point。

④ 推理与验证

在推理过程中, 激活值也按照同样的 scale 转换, 计算完成后反量化为浮点输出, 再进入下一层。

分组量化

GPTQ 会将模型的权重划分成多个小组 (group)，每组单独进行量化，这种做法的好处是：

- 1. **精度更高** —— 小范围内的权重差异更容易被保留；
- 2. **速度更快** —— 每组可以并行处理，提升量化效率。

例如，每 128 个通道为一组进行量化。

误差补偿机制

量化过程中会不可避免地引入误差。GPTQ 会在每一组量化后计算误差，并在下一组中进行修正和补偿。这种“逐组修正”的方式可以显著降低累计偏差，使最终的模型性能更接近原模型。

执行流程

加载模型 → 校准样本前向传播 → 统计权重分布 → 分组量化 → 误差补偿 → 保存模型

GPTQ 核心思想:让同伴帮忙补偿误差

传统 RTN(四舍五入)的问题

每个权重独立四舍五入,误差各自承担



✗ 总误差持续累积,精度损失大

GPTQ 的做法

产生误差后,让未量化的同伴来分担



✓ 误差被吸收,整体输出几乎无损



一个生动的类比

团队里一个人请假,其他同事分担他的工作量,让整体产出不受影响。

谁来分担

看 Hessian(敏感度矩阵)

分担多少

按数学公式精确计算

何时分担

在量化顺序中同步进行

GPTQ 三步算法流程

逐列量化并把误差实时转嫁给未处理的列,整层输出几乎等于原始 FP16

01

喂校准数据

拿约 128 条样本过一遍模型,记录每一层的输入激活,计算 Hessian 矩阵

$$H = 2XX^T$$

02

逐列量化 + 补偿

从左到右处理权重矩阵的每一列,将量化误差按 Hessian 逆矩阵指导分摊给右侧未量化列

误差转嫁公式

03

Cholesky 稳定化

用 Cholesky 分解替代迭代更新,数值稳定且 GPU 友好

175B 模型单卡数小时完成



效果:OPT-175B / BLOOM-176B 等模型量化到 4-bit,困惑度(PPL)增加 < 0.1,几乎无损

3-bit 也能基本可用· 一张 A100 几小时量化 175B 大模型

关键观察

- 大模型中只有约 1% 的权重是重要权重,它们决定了大部分性能。
- 只要保护好这 1%,其余 99% 量化到 INT4 几乎不损失精度。
- 判断权重重要性,应该看**激活值幅度**,而不是权重本身。 $y = w \cdot x$, 若输入 x 很大,即使 w 不大,量化误差也会被 x 放大。
- 真正重要的权重,是那些**对应激活值大的**。

缩放变换

- 重要权重放大 s 倍: $w \rightarrow s \cdot w$, 对应输入缩小 s 倍: $x \rightarrow x/s$, 结果不变: $s \cdot w \cdot x/s = w \cdot x$
- 但放大后的权重在量化时**相对误差显著降低**

AWQ 的核心洞察

核心观察:大模型中只有约 1% 的权重是 "重要权重",它们决定了大部分性能

✗ 错误认知

权重数值大 = 重要权重

直觉上似乎合理,但实验证明完全错误。按权重幅度挑选的"重要权重"做特殊保护,精度提升非常有限。

✓ AWQ 的发现

激活值大 = 重要权重

$y = w \cdot x$, 当 x 很大时,即使 w 不大,量化误差也会被放大。真正重要的是对应的输入激活幅度。

AWQ 的巧妙技巧:放大再量化

Step 1

找到重要权重

观察激活幅度,不看权重本身

Step 2

放大 s 倍量化

$w \rightarrow s \cdot w$, 输入 $x \rightarrow x/s$

Step 3

数学等价保证

$s \cdot w \cdot x/s = w \cdot x$, 输出不变

AWQ vs GPTQ:两种设计哲学

两者同为训练后量化,但一个追求"事后修正",一个追求"事前保护"



事后补救派

先按常规量化,事后用数学手段(Hessian 逆矩阵)修正产生的误差,把误差转嫁给未量化的权重来吸收。

- 数学派:强依赖二阶导数信息
- 计算量较大:需计算 Hessian 逆
- 精度上限高,对校准数据敏感



事前保护派

量化前就识别出重要权重,通过缩放让它们在量化时损失最小,从源头避免误差产生。

- 统计派:只看激活幅度统计
- 计算量很小:几分钟完成量化
- 泛化性好,硬件内核更高效

关键指标对比

维度	GPTQ	AWQ
量化速度	较慢(175B 约数小时)	很快(比 GPTQ 快 3× 以上)
推理速度	良好	更快(内核简单)
校准数据依赖	较敏感	不敏感(泛化好)

实践：大模型量化方法

- 实践 1：GPTQ 量化（基于梯度的训练后量化）
- 实践 2：AWQ 量化（激活感知量化）
- 综合练习项目

详细内容请参看练习手册